

**Методичні вказівки до виконання лабораторних робіт
з дисципліни: Поглиблене вивчення JavaScript**

**Лабораторна робота №3
Розробка менеджера задач із глобальним станом**

3.1 Мета роботи

Формування практичних навичок організації глобального стану у Vue-застосунках за допомогою бібліотеки Pinia, опанування маршрутизації через Vue Router, розуміння принципів масштабування компонентної архітектури та побудови багатосторінкового SPA з централізованим керуванням даними.

3.2 Загальні відомості

У попередніх лабораторних роботах стан застосунку зберігався безпосередньо у компонентах. Такий підхід зручний для простих застосунків, однак зі зростанням кількості компонентів виникає проблема передачі даних між ними. Якщо компонент-нащадок потребує даних із кореневого компонента, їх необхідно передавати через усі проміжні компоненти у вигляді props — навіть якщо вони самі ці дані не використовують. Така ситуація називається prop drilling і значно ускладнює підтримку коду.

Рішенням є використання глобального сховища стану — окремого модуля, що зберігає дані, доступні будь-якому компоненту застосунку напряму, без передачі через props. У Vue 3 стандартним інструментом для цього є бібліотека Pinia.

Окрім цього, сучасні SPA-застосунки складаються з кількох «сторінок» — різних представлень, між якими переміщається користувач. Управління цими переходами без перезавантаження браузера забезпечує Vue Router.

У межах цієї лабораторної роботи створюється повноцінний менеджер задач із підтримкою категорій, пріоритетів та дедлайнів, де навігація між списком задач, деталями та формою редагування реалізована через маршрутизацію, а всі дані зберігаються у глобальному Pinia-сховищі.

3.3 Теоретичні відомості

Передача даних через props є правильним підходом для безпосередньо пов'язаних компонентів. Однак коли дані потрібні компонентам на різних рівнях дерева, передача через кілька проміжних компонентів робить код крихким та важким для рефакторингу. Глобальне сховище вирішує цю проблему: будь-який компонент може зчитати або змінити стан напряму, звернувшись до сховища. Це спрощує архітектуру, але вимагає дисципліни — зміни стану мають відбуватись лише через визначені методи (actions), а не довільно.

Pinia є офіційно рекомендованим інструментом керування станом для Vue 3. Сховище (store) визначається як окремий файл і складається з трьох частин: state — реактивні дані сховища, аналог data у компоненті; getters — обчислювані властивості на основі state, аналог computed; actions — методи для зміни state та виконання асинхронних операцій, аналог methods. Сховище підключається до застосунку один раз, після чого будь-який компонент може отримати до нього доступ через функцію useStore. Зміни у сховищі є реактивними — всі компоненти, що використовують його дані, автоматично оновлюються.

Vue Router — офіційна бібліотека маршрутизації для Vue. Маршрут (route) — це відповідність між URL-адресою та компонентом, який потрібно відобразити. Конфігурація маршрутів визначається масивом об'єктів, де кожен об'єкт містить шлях (path) та компонент (component). Динамічні сегменти шляху записуються через двокрапку, наприклад `/tasks/:id`, де `id` є параметром, доступним у компоненті через `useRoute`. Перехід між маршрутами здійснюється через компонент `RouterLink` або програмно через `useRouter`. Поточний маршрут та його параметри доступні через хук `useRoute`.

У застосунку з маршрутизацією кореневий компонент `App.vue` містить навігаційну панель та компонент `RouterView` — місце, де відображається компонент поточного маршруту. Окремі «сторінки» реалізуються як звичайні Vue-компоненти і розміщуються у каталозі `views` або `pages`. Спільні елементи інтерфейсу — шапка, навігація — залишаються у `App.vue` і відображаються на всіх сторінках.

За замовчуванням стан `Pinia`-сховища зберігається лише в пам'яті і скидається при перезавантаженні сторінки. Для збереження між сесіями використовується `localStorage` — вбудований у браузер механізм зберігання даних у вигляді рядків. Щоразу при зміні стану його серіалізована версія записується у `localStorage`, а при ініціалізації сховища — зчитується назад. Цей підхід можна реалізувати вручну через `watch` або за допомогою плагіна `pinia-plugin-persistedstate`.

При додаванні та редагуванні задач необхідно перевіряти коректність введених даних: чи заповнені обов'язкові поля, чи відповідає дедлайн допустимому діапазону дат. Валідація виконується перед збереженням і у разі помилки відображає відповідне повідомлення поруч із полем. Стан помилок зберігається у реактивному об'єкті та очищається після успішного збереження.

3.4 Інструкція з встановлення залежностей

Для виконання лабораторної роботи необхідно додати до проекту дві бібліотеки. Встановлення виконується у каталозі проекту командами:

```
npm install pinia
npm install vue-router
```

Після встановлення обидві бібліотеки підключаються у файлі `main.js`: створюється екземпляр `router` із визначеними маршрутами та екземпляр `pinia`, обидва передаються до застосунку через метод `use`. Порядок підключення не є критичним, однак рекомендується підключати `router` перед `pinia`.

3.5 Порядок виконання роботи

Виконання лабораторної роботи здійснюється у такій послідовності:

1. Підготувати проєкт та встановити залежності:
 - a. встановити бібліотеки `pinia` та `vue-router`
 - b. підключити обидві бібліотеки у файлі `main.js`
 - c. очистити стандартний шаблон застосунку
2. Налаштувати Vue Router:
 - a. створити файл конфігурації `router/index.js`
 - b. визначити маршрути: `/` (список задач), `/tasks/:id` (деталі задачі), `/tasks/new` (створення), `/tasks/:id/edit` (редагування), `/stats` (статистика)

- c. додати компонент RouterView у App.vue
 - d. реалізувати навігаційну панель із посиланнями через RouterLink
3. Створити Pinia-сховище задач (stores/taskStore.js):
- a. визначити state: масив задач, кожна з яких містить унікальний id, назву, опис, категорію, пріоритет, дедлайн та статус виконання
 - b. реалізувати getters: список активних задач, виконаних задач, прострочених задач, кількість задач по категоріях
 - c. реалізувати actions: додавання задачі, оновлення задачі, видалення задачі, зміна статусу виконання
4. Реалізувати сторінку списку задач (views/TaskListView.vue):
- a. відобразити список задач із відображенням назви, категорії, пріоритету та дедлайну
 - b. реалізувати фільтрацію за категорією та пріоритетом одночасно через computed
 - c. реалізувати пошук за назвою задачі
 - d. реалізувати сортування за дедлайном або пріоритетом
 - e. забезпечити візуальне виділення прострочених задач
 - f. додати кнопку переходу до створення нової задачі
5. Реалізувати сторінку деталей задачі (views/TaskDetailView.vue):
- a. отримати id задачі з параметрів маршруту через useRoute
 - b. знайти відповідну задачу у сховищі за id
 - c. відобразити всі поля задачі
 - d. додати кнопки переходу до редагування та видалення
 - e. при видаленні — перенаправити користувача на список задач через useRouter
 - f. обробити випадок, коли задачу з таким id не знайдено
6. Реалізувати форму створення та редагування задачі (views/TaskFormView.vue):
- a. форма використовується як для створення (/tasks/new), так і для редагування (/tasks/:id/edit)
 - b. при редагуванні — заповнити поля форми даними існуючої задачі
 - c. реалізувати валідацію: назва є обов'язковою, дедлайн не може бути у минулому при створенні
 - d. при збереженні — викликати відповідний action сховища та перенаправити користувача
7. Реалізувати сторінку статистики (views/StatsView.vue):
- a. відобразити загальну кількість задач та розбивку за статусом
 - b. відобразити список найближчих дедлайнів — задач, що завершуються протягом 3 днів
 - c. відобразити кількість прострочених задач
 - d. використати виключно getters зі сховища — жодної логіки у компоненті
8. Перевірити навігацію та цілісність стану:
- a. перехід між сторінками не скидає список задач
 - b. зміни на сторінці редагування відображаються у списку без перезавантаження
 - c. видалення задачі відображається одразу на сторінці статистики
 - d. некоректний маршрут обробляється коректно
9. Перевірити працездатність застосунку у повному обсязі:
- a. коректна робота всіх маршрутів

- b. відсутність помилок у консолі браузера
- c. коректна валідація форм
- d. узгодженість даних між усіма сторінками

Додатково (для отримання максимальної оцінки):

- 10. Реалізувати збереження стану у `localStorage`:
 - a. при кожній зміні сховища серіалізувати стан у `localStorage` через `watch` або плагін
 - b. при ініціалізації сховища зчитувати збережений стан
 - c. забезпечити коректну роботу після перезавантаження сторінки
- 11. Реалізувати анімації переходів між маршрутами:
 - a. використати компонент `Transition` разом із `RouterView`
 - b. визначити CSS-класи для анімації входу та виходу сторінки
- 12. Розширити функціонал сторінки статистики:
 - a. додати прогрес-бар виконання задач

3.6 Приклад реалізації застосунку

Застосунок складається з чотирьох основних сторінок, між якими користувач переміщається через навігаційну панель та програмні переходи. Навігаційна панель присутня на всіх сторінках, оскільки вона розміщена у кореневому компоненті `App.vue` поза компонентом `RouterView`.

`Pinia`-сховище містить масив задач як основний стан. Кожна задача є об'єктом із полями: унікальний числовий ідентифікатор, назва, опис, категорія (рядок), пріоритет (один із трьох рівнів), дедлайн (рядок у форматі дати) та булевий прапорець виконання. Ідентифікатор при створенні генерується на основі поточного часу або лічильника, щоб гарантувати унікальність.

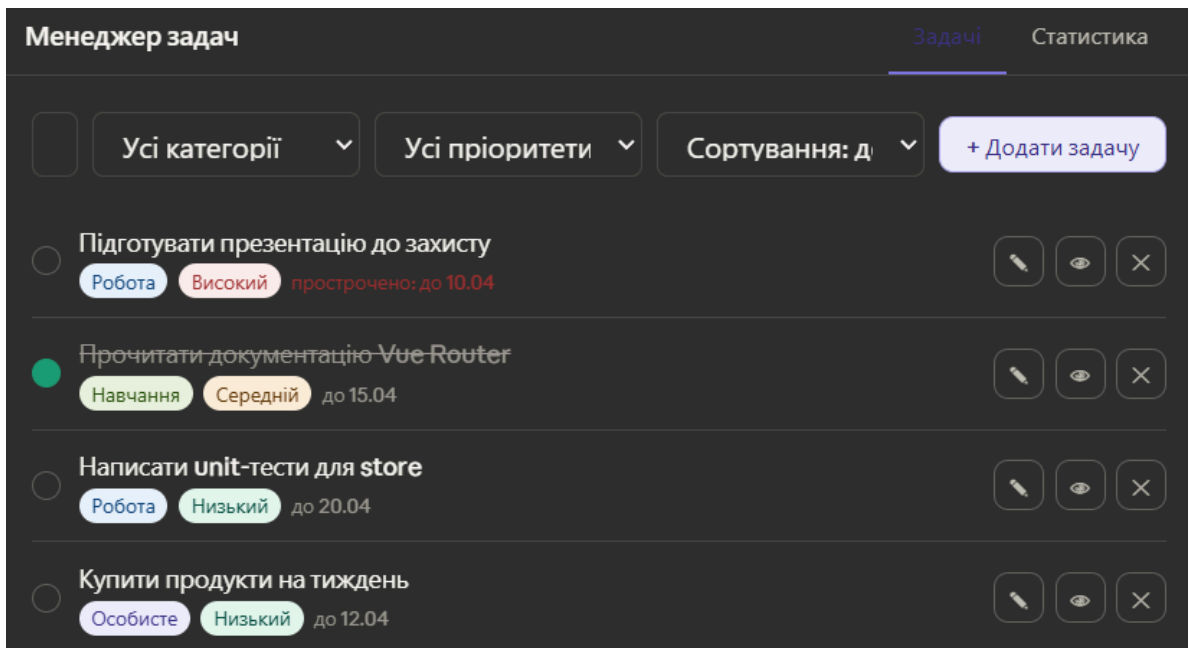
`Getters` сховища формують похідні колекції: список лише активних задач, список виконаних, список прострочених (дедлайн яких менший за поточну дату і які ще не виконані), а також список задач із найближчими дедлайнами. Усі ці дані доступні будь-якому компоненту без дублювання логіки.

Сторінка списку використовує `computed` для формування відфільтрованого та відсортованого масиву на основі поточних значень фільтрів та пошукового рядка. Самі фільтри є локальним станом компонента — вони не потребують глобального сховища, оскільки актуальні лише для цієї сторінки.

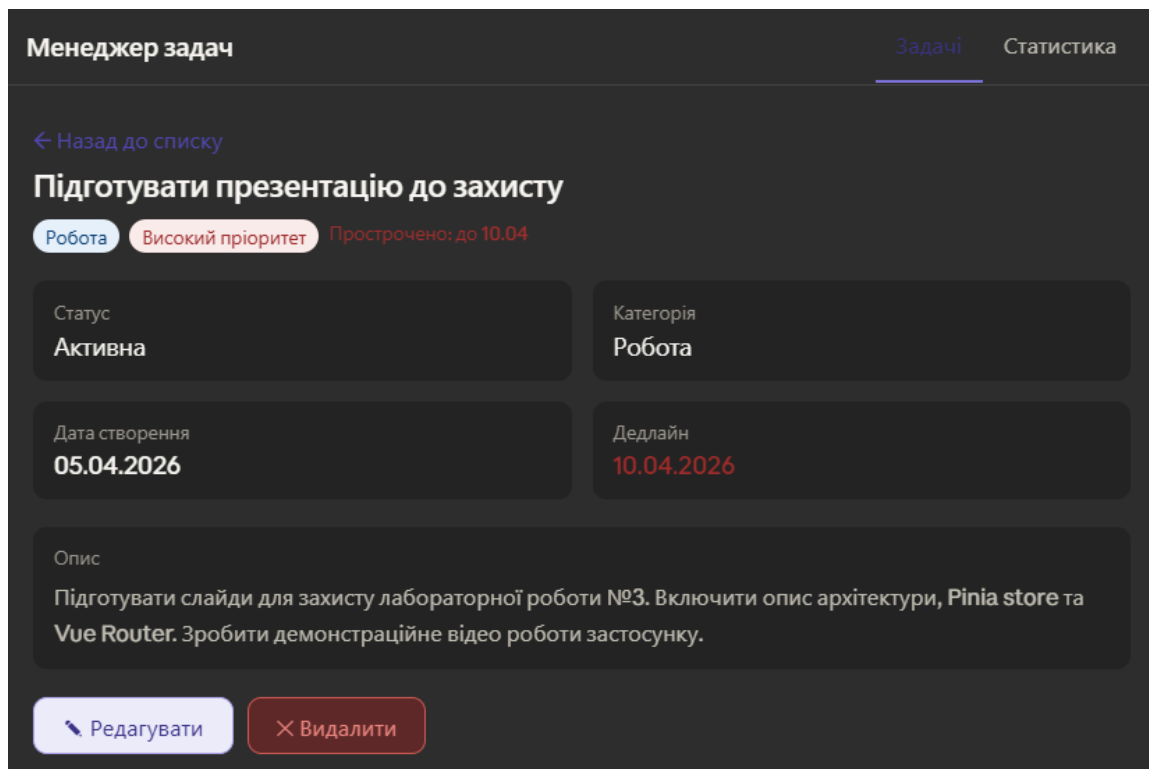
Сторінка деталей отримує параметр `id` із поточного маршруту та знаходить задачу у сховищі. Якщо задачу не знайдено, відображається повідомлення про помилку або відбувається автоматичне перенаправлення на список.

Форма створення та редагування є спільним компонентом для обох операцій. Логіка визначення режиму (створення чи редагування) базується на наявності параметра `id` у маршруті. При редагуванні поля форми ініціалізуються даними існуючої задачі, а при збереженні викликається відповідний `action`.

Приклад реалізації сторінки зі списком задач:



Можливість перегляду деталей задач:



Редагування задачі:

Менеджер задач Задачі Статистика

[← Назад до деталей](#)

Редагування задачі

Назва задачі

Підготувати презентацію до захисту

Опис

Підготувати слайди для захисту лабораторної роботи №3...

Категорія

Робота

Пріоритет

Високий

Дедлайн

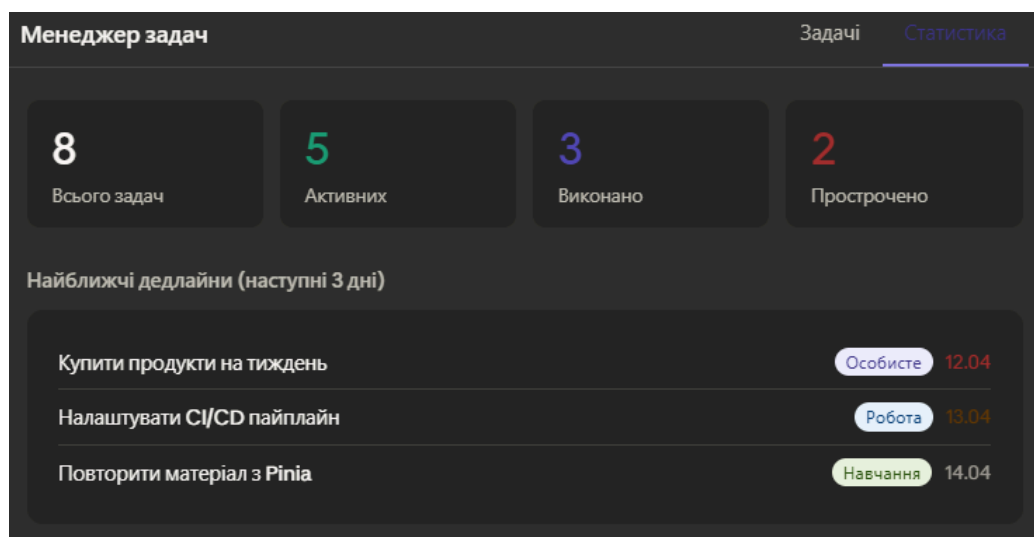
10/04/2026

Статус

Активна

Скасувати Зберегти зміни

Сторінка статистики:



3.7 Завдання до виконання

У процесі виконання лабораторної роботи необхідно:

- встановити та підключити бібліотеки Pinia і Vue Router
- налаштувати маршрутизацію з п'ятьма маршрутами
- створити Pinia-сховище з state, getters та actions для керування задачами
- реалізувати сторінку списку задач з фільтрацією за категорією, пріоритетом та пошуком за назвою
- реалізувати сторінку деталей задачі з можливістю видалення
- реалізувати форму створення та редагування задачі з валідацією
- реалізувати сторінку статистики на основі getters сховища

- забезпечити узгодженість стану між усіма сторінками

3.8 Вимоги до захисту

Усі лабораторні роботи виконуються у вигляді окремих проєктів або модулів одного проєкту. Код має бути структурованим, із чітким поділом на компоненти, views та stores, з дотриманням принципів чистого коду.

Результати роботи студент повинен представити у вигляді:

- вихідного коду
- короткого звіту з описом мети, теоретичних відомостей та реалізації
- демонстрації працездатності застосунку

Під час захисту особлива увага приділяється розумінню різниці між локальним та глобальним станом, логіці організації маршрутизації, структурі Pinia-сховища та вмінню пояснити власний код.

3.9 Оцінювання лабораторної роботи

Лабораторна робота оцінюється максимально у 25 балів. Базовий функціонал (маршрутизація, сховище, CRUD операції, фільтрація, статистика) оцінюється у 20 балів. Додаткові завдання (збереження у localStorage, анімації переходів, розширена статистика) дають змогу отримати решту балів. Оцінка також залежить від якості коду та здатності студента пояснити реалізацію під час захисту.

3.10 Контрольні питання

1. Що таке prop drilling і яку проблему він спричиняє?
2. З яких трьох частин складається Pinia-сховище? Яке призначення кожної?
3. Як компонент отримує доступ до даних Pinia-сховища?
4. Що таке маршрут у Vue Router? Як визначаються динамічні параметри маршруту?
5. Яке призначення компонентів RouterView та RouterLink?
6. Як отримати параметри поточного маршруту у компоненті?
7. Як програмно перенаправити користувача на інший маршрут?
8. Чим відрізняються getters від actions у Pinia-сховищі?
9. Як реалізувати збереження стану сховища між перезавантаженнями сторінки?
10. Яку перевагу дає винесення логіки фільтрації у getters сховища порівняно з реалізацією у кожному компоненті окремо?